

## Design Documentation

### Low-Level Design (LLD)

Instacart Market Basket Analysis — Python, MySQL, Database, and Tableau Pipeline

## 1. Project Directory Structure

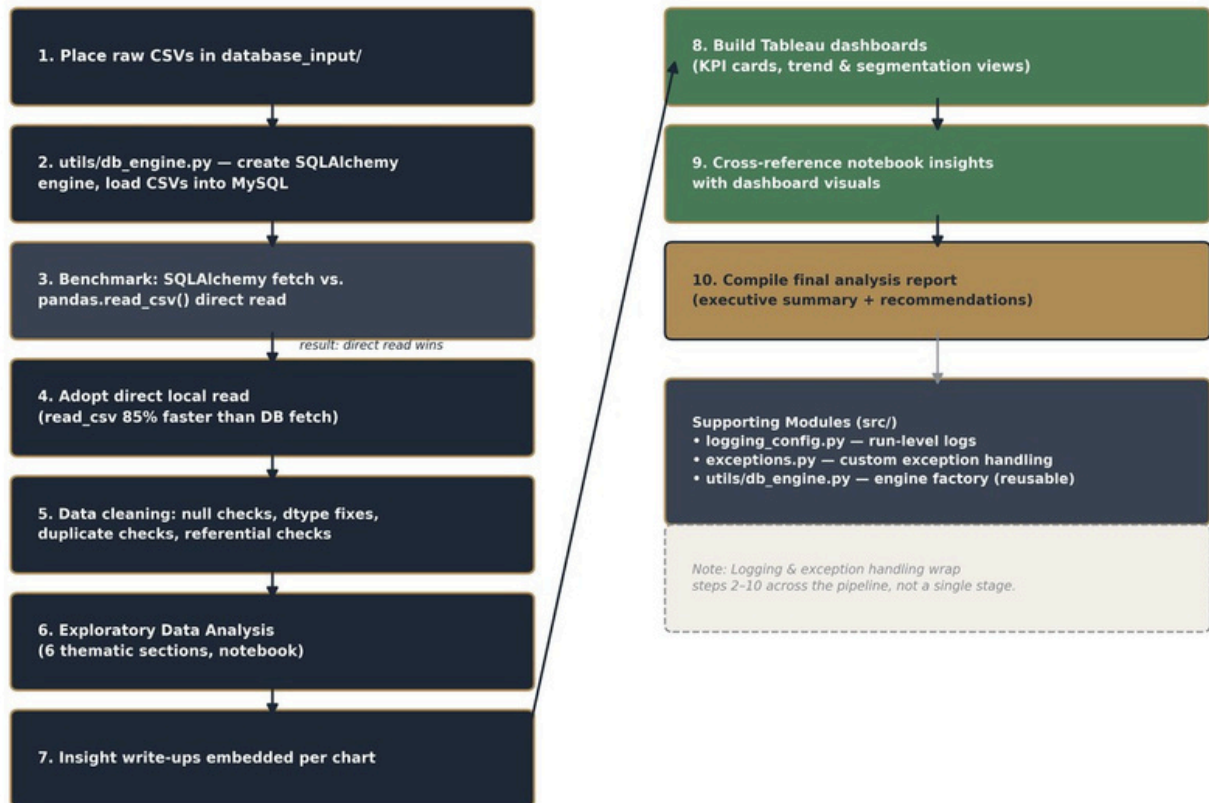


Figure 2. Repository layout showing source, utility, notebook, and output directories.

## 2. Detailed Process Flow

## Low-Level Design — Process Flow

*Step-by-step execution flow with supporting modules*



*Figure 3. Step-by-step execution flow from raw file to final report.*

### 3. Module-Level Detail

| Module                            | Description                                                                                                                                         |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| src/utils/engine_creator function | Factory function that builds and returns a configured SQLAlchemy engine for MySQL connections; reused wherever a database connection is needed.     |
| src/data_ingestion.py             | Reads CSVs from database_input/ and writes them into MySQL tables via the engine factory.                                                           |
| src/logging_config.py             | Centralized logging configuration applied across ingestion and processing scripts; writes run-level logs to logs/pipeline.log.                      |
| src/exceptions.py                 | Custom exception classes used to raise and handle pipeline-specific errors (e.g. missing file, schema mismatch) distinctly from generic exceptions. |
| notebooks/eda_analysis.ipynb      | Primary analysis notebook: data cleaning, error checks, and EDA across six thematic sections, with insights recorded inline.                        |
| tableau/dashboard.twbx            | KPI and segmentation dashboards built from the cleaned dataset.                                                                                     |
| reports/final_report.docx         | Consolidated report combining notebook insights and dashboard findings into a single business-facing deliverable.                                   |

## 4. Data Ingestion and Access — Detailed Steps

- Raw CSV files (aisles, departments, orders, products, order\_products\_prior) are placed in database\_input/.
- The SQLAlchemy engine factory (utils/engine\_creator) establishes a connection to MySQL using connection parameters.
- data\_ingestion.py loads each CSV into a corresponding MySQL table for the database-backed path.
- A benchmark comparison is run: fetching data back out via SQLAlchemy versus reading the original CSVs directly with pandas.read\_csv().
- Direct local read is adopted as the production data access method for all downstream analysis, based on the approximately 85% performance improvement observed.

## 5. Data Cleaning and Error Checking

- Null value checks across all source tables, with explicit handling for expected nulls (e.g. days\_since\_prior\_order is null on a customer's first order by design, not a data quality issue).
- Data type validation and correction (e.g. numeric fields, categorical encodings).

- Duplicate record checks on primary identifiers (order\_id, product\_id).
- Referential checks confirming that foreign keys (aisle\_id, department\_id, product\_id) resolve correctly across joined tables.

## 6. Exploratory Data Analysis Structure

The EDA notebook is organized into six thematic sections, each containing one or more business questions, a supporting chart, and an inline markdown insight summarizing the finding:

- Section 1 — Product and Category Performance
- Section 2 — Customer Purchase Behavior
- Section 3 — Temporal and Seasonality Patterns
- Section 4 — Reorder Behavior
- Section 5 — Department and Aisle Level Segmentation

Each chart is immediately followed by a formatted insight block interpreting the result in business terms, rather than restating the chart's values.

## 7. Logging and Exception Handling

Logging and exception handling are cross-cutting concerns applied throughout the ingestion and processing steps rather than confined to a single stage:

- logging.py defines a consistent log format and writes run-level events (start/end of ingestion, row counts, warnings) to logs/pipeline.log.
- exceptions.py defines custom exception types so pipeline-specific failures (e.g. a missing source file, a schema mismatch during ingestion) can be caught and handled distinctly from generic Python exceptions.
- Both are imported and used within data\_inhibitor.py and the notebook's data-loading cells to ensure failures are logged and surfaced clearly rather than failing silently.

## 8. Visualization and Reporting

- Tableau dashboards are built from the cleaned dataset and organized around KPI cards (e.g. reorder rate, average basket size) and segmentation views (department, aisle, time-of-day).
- Dashboard findings are cross-referenced against the notebook's EDA insights to ensure consistency between the two views of the data.
- The final report consolidates both sources into a single business-facing document, structured with an executive summary, section-by-section findings, and prioritized recommendations.